

## LAB 2: TRIẾT LÝ REACTJS, VIRTUAL DOM, JSX VÀ KHỞI TẠO HỆ THỐNG VỚI VITE

### I. MỤC TIÊU BÀI THỰC HÀNH

Sau khi hoàn thành bài thực hành này, người học có khả năng:

- **Hiểu và vận dụng triết lý ReactJS:** Thành thạo tư duy khai báo (Declarative programming) và kiến trúc dựa trên cấu phần (Component-based architecture).
- **Làm chủ cấu trúc dự án hiện đại:** Khởi tạo, cấu hình và vận hành dự án ReactJS bằng công cụ biên dịch tối ưu Vite.
- **Thành thạo cú pháp JSX:** Kết hợp chặt chẽ giữa mã logic JavaScript và cấu trúc giao diện XML để hiển thị dữ liệu động.
- **Trực quan hóa cơ chế Virtual DOM:** Kiểm chứng thuật toán đối chiếu (Reconciliation) thông qua việc thao tác trực tiếp trên các phần tử danh sách có sử dụng thuộc tính định danh key.

### II. CƠ SỞ LÝ THUYẾT TRỌNG TÂM

Trước khi bắt đầu các bước cấu hình và lập trình, lập trình viên cần nắm vững mô hình lý thuyết tổng quát của ReactJS:

1. **Hàm số hóa Giao diện (UI as a Function):** Bản chất của triết lý Khai báo quy định giao diện hiển thị đơn giản là kết quả đầu ra của một hàm số nhận dữ liệu đầu vào, biểu diễn bằng công thức toán học sơ cấp:  $UI = f(state, props)$ . Khi dữ liệu thay đổi, hàm số tự động tính toán lại mà không cần sự can thiệp thủ công vào cây cấu trúc của trình duyệt.
2. **Thuật toán đối chiếu Virtual DOM (Reconciliation):** Khi có sự thay đổi về mặt trạng thái (state), React xây dựng một cây đối tượng JavaScript ảo trong bộ nhớ đệm (RAM). Nó tiến hành chạy thuật toán Heuristic có độ phức tạp tuyến tính  $O(n)$  để so sánh sai khác với phiên bản cũ, sau đó chỉ cập nhật (patch) những phần tử bị biến đổi lên cây DOM thật của trình duyệt (Real DOM).

### III. NỘI DUNG THỰC HÀNH CHI TIẾT

#### Phần 1: Cấu hình môi trường và khởi tạo dự án với Vite

Thay vì sử dụng các công cụ đóng gói truyền thống vốn khởi động chậm chạp do phải đóng gói toàn bộ mã nguồn trước khi chạy server, chúng ta sử dụng **Vite**. Vite tận dụng cơ chế Native ESM trên các trình duyệt hiện đại giúp biên dịch mã nguồn theo yêu cầu (on-demand), tối ưu hóa tốc độ phát triển.

**Bước 1.1: Mở cửa sổ Terminal tại thư mục làm việc mong muốn và thực thi câu lệnh khởi tạo dự án với template React thuần mã JavaScript:**

```
npm create vite@latest lab-react-philosophy -- --template react
```

**Bước 1.2: Di chuyển vào thư mục dự án vừa tạo và cài đặt toàn bộ các thư viện phụ thuộc bắt buộc (dependencies) được liệt kê trong file package.json:**

```
cd lab-react-philosophy
npm install
```

**Bước 1.3: Kích hoạt máy chủ phát triển cục bộ (Development Server) bằng lệnh:**

```
npm run dev
```

*Mẹo nhỏ: Quan sát màn hình Terminal để lấy địa chỉ URL cục bộ (mặc định là `http://localhost:5173/`). Mở trình duyệt Chrome và truy cập vào địa chỉ này để đảm bảo hệ thống đã hoạt động bình thường.*

**Bước 1.4: Tìm hiểu cấu trúc thư mục:** Dùng VS Code mở thư mục dự án. Hãy chú ý các tệp tin cốt lõi:

- `index.html`: Nằm tại thư mục gốc của dự án. Chứa thẻ `<div id="root"></div>` – nơi toàn bộ ứng dụng React sẽ được gắn vào.
- `src/main.jsx`: Điểm khởi đầu (Entry point) của ứng dụng. Tập này sử dụng phương thức `ReactDOM.createRoot` để liên kết cấu phần gốc với thẻ `div` trong file HTML.
- `src/App.jsx`: Cấu phần gốc (Root Component) của ứng dụng, nơi chúng ta sẽ viết mã thực hành chính.

#### Phần 2: Thực hành cú pháp JSX và Triết lý Component-based

Mục tiêu của phần này là xây dựng một danh sách thông tin sinh viên động, áp dụng tư duy Component-based (chia nhỏ giao diện) và nhúng biểu thức logic trực tiếp vào cấu trúc HTML bằng JSX.

**Bước 2.1: Tạo mới một tệp tin cấu phần tại đường dẫn `src/StudentCard.jsx` để định nghĩa khung hiển thị cho từng sinh viên đơn lẻ. Sao chép đoạn mã chuẩn hóa sau:**

```
import React from 'react';

function StudentCard({ student }) {
  // Triết lý Khai báo: Điều kiện hóa giao diện dựa trên dữ liệu đầu vào
  return (
    <div style={{
      border: '1px solid #cbd5e0',
      borderRadius: '8px',
      padding: '15px',
      marginBottom: '10px',
      backgroundColor: student.isActive ? '#f0fff4' : '#fff5f5'
    }}>
      <h3 style={{ margin: '0 0 8px 0', color: '#2d3748' }}>{student.name}</h3>
      <p style={{ margin: '0 0 5px 0' }}><strong>Mã số:</strong> {student.code}</p>
      <p style={{ margin: '0' }}>
        <strong>Trạng thái học tập:</strong> { ' ' }
        <span style={{ color: student.isActive ? '#38a169' : '#e53e3e', fontWeight:
'bold' }}>
          {student.isActive ? 'Đang học' : 'Đã tốt nghiệp / Tạm dừng'}
        </span>
      </p>
    </div>
  );
}

export default StudentCard;
```

**Bước 2.2: Cập nhật tệp tin `src/App.jsx` để làm nhiệm vụ quản lý nguồn dữ liệu (State) và phân phối dữ liệu xuống cấu phần con thông qua cơ chế luồng dữ liệu một chiều (Unidirectional Data Flow):**

```
import React, { useState } from 'react';
import StudentCard from './StudentCard';

function App() {
  // Khởi tạo nguồn chân lý duy nhất (Single Source of Truth) cho dữ liệu
  const [students, setStudents] = useState([
    { id: 'st01', code: 'SV001', name: 'Nguyễn Văn A', isActive: true },
    { id: 'st02', code: 'SV002', name: 'Trần Thị B', isActive: false },
    { id: 'st03', code: 'SV003', name: 'Lê Hoàng C', isActive: true }
  ]);

  return (
    <div style={{ maxWidth: '600px', margin: '30px auto', padding: '0 20px' }}>
      <h1 style={{ textAlign: 'center', color: '#1a365d' }}>Quản Lý Sinh Viên</h1>
      <div style={{ marginTop: '20px' }}>
        /* Vận dụng biểu thức JSX lặp mảng bằng hàm toán học .map() */
        {students.map((student) => (
```

```
        <StudentCard key={student.id} student={student} />
      )}}
    </div>
  </div>
);
}

export default App;
```

*Phân tích triết lý: Ở đoạn code trên, mã HTML không đứng độc lập mà lồng ghép trực tiếp với phương thức xử lý mảng của JavaScript thông qua cặp dấu ngoặc nhọn {}. Dữ liệu chạy từ App xuống StudentCard dưới dạng thuộc tính (props). Cấu phần con hoàn toàn không thể chỉnh sửa mảng gốc, đảm bảo tính nhất quán.*

### Phần 3: Trục quan hóa Virtual DOM và Tầm quan trọng của thuộc tính Key

Phần này giúp người học hiểu rõ cách thức hoạt động của Virtual DOM trên trình duyệt và tại sao React yêu cầu nghiêm ngặt thuộc tính key duy nhất khi xử lý mảng.

**Bước 3.1: Bổ sung một tính năng cho phép thêm một sinh viên mới vào vị trí ĐẦU MẢNG trong file src/App.jsx. Hãy cập nhật đoạn code của hàm App như sau:**

```
// Thêm phương thức xử lý sự kiện vào bên trong hàm App
const addStudentAtTop = () => {
  const newStudent = {
    id: `st_${Date.now()}`,
    code: `SV${Math.floor(100 + Math.random() * 900)}`,
    name: `Sinh Viên Mới (${new Date().toLocaleTimeString()}`,
    isActive: true
  };

  // Áp dụng tính bất biến (Immutability) để cập nhật trạng thái mới
  setStudents([newStudent, ...students]);
};

// Cập nhật phần return của App để bổ sung nút bấm hành động
return (
  <div style={{ maxWidth: '600px', margin: '30px auto', padding: '0 20px' }}>
    <h1 style={{ textAlign: 'center', color: '#1a365d' }}>Quản Lý Sinh Viên</h1>
    <button onClick={addStudentAtTop} style={{ marginBottom: '20px' }}>
      Thêm Sinh Viên Vào Đầu Danh Sách
    </button>
    <div style={{ marginTop: '20px' }}>
      {students.map((student) => (
        <StudentCard key={student.id} student={student} />
      ))}
    </div>
  </div>
);
```

### **Bước 3.2: Thực hiện kiểm chứng kỹ thuật trên trình duyệt (Chrome DevTools):**

1. Mở ứng dụng trên trình duyệt Chrome, nhấn phím F12 (hoặc chuột phải chọn **Inspect**) để bật DevTools.
2. Chuyển qua tab **Elements**, mở rộng cây thư mục HTML cho đến khi nhìn thấy danh sách các thẻ div của StudentCard.
3. Nhấn nút "*Thêm Sinh Viên Vào Đầu Danh Sách*" trên màn hình và quan sát kỹ hiệu ứng nhấp nháy sáng (Flash update) của Chrome DevTools trên các nút DOM:
  - o **Trường hợp có `key={student.id}`**: Bạn sẽ thấy CHỈ CÓ DUY NHẤT một thẻ div mới ở trên cùng nhấp nháy sáng lên. Các thẻ div sinh viên cũ hoàn toàn giữ nguyên trạng thái tĩnh và chỉ bị đẩy dịch xuống dưới. Điều này minh chứng Virtual DOM đã tính toán chuẩn xác và giảm thiểu thao tác vẽ lại (Repaint) đối với các phần tử cũ.

*Thử nghiệm tiêu cực để hiểu bản chất: Hãy thử xóa bỏ thuộc tính `key={student.id}` trong hàm `.map()` hoặc thay thế bằng chỉ số mảng thông thường `key={index}`. Tiến hành nhấn nút thêm phần tử và quan sát tab Elements. Bạn sẽ thấy TẤT CẢ các thẻ div sinh viên trong danh sách đều bị ép buộc phải nhấp nháy sáng. React lúc này không thể phân biệt được các cấu phần cũ và buộc phải dựng lại toàn bộ giao diện từ đầu, làm suy giảm hiệu năng nghiêm trọng.*

## **IV. BÀI TẬP NÂNG CAO TỰ GIẢI**

Để hoàn thiện kỹ năng độc lập giải quyết bài toán, sinh viên thực hiện bổ sung tính năng sau vào mã nguồn hiện tại:

**Yêu cầu kỹ thuật:** Hãy xây dựng một cơ chế cho phép người dùng click vào một cấu phần StudentCard bất kỳ để đảo ngược trạng thái học tập của sinh viên đó (từ `isActive: true` chuyển thành `false` và ngược lại).

- Học viên phải truyền một hàm callback xử lý sự kiện từ cấu phần cha App xuống cấu phần con StudentCard.
- Thực hiện cập nhật danh sách bằng phương thức bất biến `.map()` để thay đổi duy nhất đối tượng sinh viên được chọn dựa trên tham số định danh id.

## V. TIÊU CHÍ ĐÁNH GIÁ VÀ BẢNG ĐIỂM (RUBRIC)

| Tiêu chí                             | Yêu cầu chi tiết                                                                                                                     | Trọng số điểm |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|---------------|
| <b>Khởi tạo &amp; Vận hành</b>       | Dự án được tạo lập thành công bằng cấu hình Vite; chạy mượt mà không phát sinh lỗi biên dịch trên console hệ thống.                  | 2.0 Điểm      |
| <b>Áp dụng cấu pháp JSX</b>          | Sử dụng chính xác biểu thức nhúng, toán tử ba ngôi để đổi màu nền thẻ dynamic và lặp mảng chuẩn bằng biểu thức <code>.map()</code> . | 3.0 Điểm      |
| <b>Làm chủ Virtual DOM &amp; Key</b> | Giải thích và chứng minh được cơ chế hoạt động tối ưu của Virtual DOM thông qua kiểm chứng thực tế với thuộc tính key trên DevTools. | 3.0 Điểm      |
| <b>Giải quyết bài tập nâng cao</b>   | Hoàn thiện trọn vẹn logic tính năng đảo ngược trạng thái sử dụng callback prop đúng chuẩn luồng dữ liệu một chiều của React.         | 2.0 Điểm      |